Лекция 11. Мониторинг данных и качества модели

Версия: 2026–2027 учебный год Продолжительность: 2 академических часа Связанные работы: ЛР11 «Наблюдаемость ML-сервиса», ЛР12 «Drift и отложенное качество»

1. Цель лекции

Научить студентов строить доказательный контур мониторинга модели: отделять исправность сервиса от качества прогнозов, сравнивать согласованные временные окна, учитывать задержку фактических меток и принимать управляемое решение о реакции. После лекции студент должен уметь объяснить, почему drift является сигналом расследования, но не доказательством необходимости retraining.

2. Результаты обучения

Студент сможет:

- различать data drift, prediction drift, concept drift и data quality incident;
- выбирать reference и current окна без утечки будущего;
- объяснять PSI и ограничения статистических тестов;
- вычислять label coverage только по зрелым прогнозам;
- считать качество по model version и временному окну;
- отличать отсутствие доказательств от доказательства отсутствия проблемы;
- формулировать policy и decision gate;
- связывать отчёт с release manifest, runbook и postmortem.

3. Место в сквозном проекте

В ЛР11 команда доказала, что API доступен, измерим и связан с model version. Это ещё не подтверждает полезность модели. Сервис может отвечать быстро и без 5xx, но выдавать устаревшие прогнозы. ЛР12 добавляет второй слой: характеристики входов, распределение прогнозов и качество после появления фактической задержки рейса.

На доске удобно провести границу: service health отвечает на вопрос «работает ли система», model health — «сохраняет ли результат ожидаемое качество и применимость».

4. Единица наблюдения

Минимальная запись связывает `request_id`, `event_time`, признаки, probability, решение, `model_version` и `source_commit`. Позже к ней присоединяются `label_time` и фактический исход. Полезно подчеркнуть: без устойчивого идентификатора и времени нельзя надёжно соединить прогноз с меткой, а без версии модели нельзя понять, какой release деградировал.

Сырые признаки не обязаны попадать в оперативные логи. Их хранение должно соответствовать политике приватности. Для учебного кейса используются обезличенные `DepHour` и `IsWeekend`.

5. Data quality incident не равен drift

Если `DepHour` внезапно принимает значение 900, сначала нарушен контракт диапазона. Если столбец перестал поступать — это schema/data quality incident. Drift описывает изменение допустимого распределения, например рост доли вечерних рейсов. Сначала применяются контрактные проверки, затем статистический мониторинг. Иначе поломка источника будет ошибочно названа естественным изменением.

6. Четыре разных сигнала

Сигнал	Что изменилось	Нужны labels	Что доказывает
Data drift	распределение X	нет	входы отличаются от reference
Prediction drift	распределение score или класса	нет	поведение модели изменилось
Concept drift	условное распределение Y при X	обычно да	связь входов и результата изменилась
Quality degradation	метрики по Y и prediction	да	результат хуже принятого порога

Один сигнал не подменяет другой. Prediction drift может быть нормальной реакцией на сезонность, а качество может ухудшиться без заметного marginal drift отдельных признаков.

7. Reference-окно

Reference должно быть связано с решением: обучающая часть, validation перед выпуском или стабильное production-окно champion-модели. Нельзя незаметно заменять reference после каждого предупреждения — так система «забывает» исходную норму. Версия reference, границы времени, фильтры и model version фиксируются в policy и отчёте.

8. Current-окно

Current-окно выбирается по частоте решений и объёму потока: сутки, неделя или последние N событий. Слишком короткое окно шумит, слишком длинное скрывает резкое изменение. Для каждой проверки нужны начало, конец, число строк и правило включения. Окна сравнивают одинаковую семантику признаков.

9. Хронология и утечка

Все границы строятся по event time, а не по удобному порядку строк. Reference не должен содержать события после current. При backfill отчёт сохраняет явный `as_of`, иначе результат невозможно воспроизвести. Фактическая задержка рейса никогда не используется как входной признак.

10. Population Stability Index

Для категорий или bins используются доли reference `p_i` и current `q_i`:

`PSI = sum((q_i - p_i) * ln(q_i / p_i))`.

Нулевые доли сглаживаются малым epsilon. Границы числовых bins строятся только по reference, иначе current влияет на измерительную шкалу. В учебной policy значения 0.10 и 0.25 служат warning и alert, но это договорные пороги, а не универсальный закон.

11. Ограничения PSI

PSI зависит от binning, размера окна и редких категорий. Он не сообщает причину и не измеряет бизнес-ущерб. При большом потоке даже небольшое отличие может быть устойчивым, но незначимым для решения; при малом потоке оценка нестабильна. Поэтому отчёт всегда содержит объём окна и метод.

12. Статистическая и практическая значимость

P-value отвечает не на вопрос о величине эффекта. При большом N тест способен найти микроскопическое отличие. Обсудите с аудиторией пару: статистический тест плюс effect size и инженерный threshold. До расчёта команда должна определить, какое изменение способно повлиять на решение или риск.

13. Многократные проверки

Если отслеживаются десятки признаков, вероятность случайного срабатывания растёт. Возможные меры: коррекция множественных сравнений, требование доли drifting features, повторное подтверждение на следующем окне или приоритизация ключевых признаков. Один случайный флаг не должен автоматически создавать новую champion-модель.

14. Prediction drift

Распределение probabilities и доля положительного класса доступны сразу. Это ранний сигнал, но он смешивает изменение входов и реакцию модели. В отчёте его связывают с model version и threshold. После изменения threshold доля положительных решений может измениться без изменения самой модели.

15. Почему labels задерживаются

Прогноз строится за 60 минут до вылета, а фактическая задержка прибытия известна после события. Метка может дополнительно задержаться из-за ETL. В кредитном риске или обслуживании оборудования лаг измеряется неделями и месяцами. Поэтому «нет метки сейчас» не означает ошибку прогноза.

16. Mature cohort

Пусть ожидаемая задержка метки равна 24 часам, а отчёт строится в момент `as_of`. В знаменатель coverage входят только прогнозы с `event_time <= as_of - 24h`. Более свежие события ещё не обязаны иметь метку и исключаются. Это центральное правило ЛР12.

17. Label coverage

`coverage = matched mature labels / eligible mature predictions`.

Coverage указывается вместе с числами числителя и знаменателя. Низкий coverage может быть вызван ошибкой join, задержкой источника, несовпадающим часовым поясом или выборочным поступлением меток. Считать F1 на небольшой смещённой подвыборке опасно даже при технически корректной формуле.

18. Join прогноза и метки

`request_id` должен быть уникальным в обоих журналах. Join проверяется как one-to-one. Время хранится в UTC. Повторная метка не должна молча дублировать прогноз и менять вес наблюдения. Для исправления истории применяется версионированный backfill, а не удаление неудобных строк.

19. Оконные метрики качества

После достижения минимального coverage рассчитываются ROC AUC, precision, recall и F1 для одной model version и одного окна. Если в mature cohort представлен только один класс, ROC AUC не определён: статус должен быть `insufficient_evidence`, а не ноль и не успех.

20. Сегменты

Общая метрика может скрывать провал сегмента: будние/выходные, часы суток, аэропорты или регионы. Сегмент включается только при достаточном N и допустимости обработки данных. Нельзя публиковать чувствительные малые группы. В ЛР12 обязателен хотя бы один безопасный учебный сегмент.

21. Decision policy

Полезная policy содержит окна, minimum rows, PSI thresholds, expected label delay, minimum coverage, quality thresholds, владельца и требование review. Пример действий:

- `hold` — доказательств недостаточно;
- `investigate` — drift без подтверждённой деградации;
- `incident_review` — качество упало без очевидного drift;
- `retraining_review` — drift и quality breach подтверждены;
- `no_action` — пороги не нарушены.

22. Почему нет automatic retraining

Переобучение на повреждённых или неполных labels закрепит дефект. Даже корректный candidate должен пройти validation, Registry gates и review ЛР10. Автоматизировать можно расчёт сигнала и создание issue; продвижение champion остаётся управляемым решением с rollback.

23. Runbook и postmortem

Runbook отвечает, что делать сейчас: проверить окна, контракт, coverage, model version и источник labels. Postmortem отвечает, почему инцидент произошёл, каков эффект и какое изменение предотвратит повторение. Фраза «переобучили модель» без анализа первопричины недостаточна.

24. Evidently и прозрачная реализация

Evidently предоставляет Reports, Presets, тесты и экспорт HTML/JSON. На 20 октября 2026 года в материалах фиксируется ветка `evidently>=0.7.23,<0.8`. Обязательная реализация курса остаётся на pandas/NumPy: студент должен понимать окна, bins, coverage и gate. Evidently используется для сравнения результатов, а версия и конфигурация сохраняются.

25. Демонстрация преподавателя

1. Запустить `python -m mlops_flight_delay.monitoring` на учебных CSV.

2. Показать `eligible_cutoff`, числитель и знаменатель coverage.
3. Изменить `as_of` так, чтобы свежие прогнозы стали mature.
4. Удалить несколько labels и получить `insufficient_evidence`.
5. Показать drift alert при хорошем качестве: решение остаётся `investigate`.
6. Показать сочетание drift и quality breach: появляется `retraining_review`, но не выпуск.

26. Вопросы аудитории

1. Почему свежие прогнозы нельзя считать отсутствующими labels?
2. Чем data quality incident отличается от data drift?
3. Почему bins должны определяться по reference?
4. Когда ROC AUC нельзя вычислить?
5. Может ли качество упасть без data drift?
6. Почему высокий drift не означает обязательное retraining?
7. Какие поля обеспечивают связь с release?
8. Что доказывает coverage и чего оно не доказывает?

27. Связь с ЛР12 и ЛР13

В ЛР12 команда создаёт version-controlled policy, JSON-отчёт, тесты, runbook и индивидуальные сценарии. В ЛР13 те же timestamps используются для point-in-time join признаков. Ошибка времени в feature join способна одновременно создать leakage при обучении и ложный monitoring signal.

28. Официальные источники

- Evidently PyPI: https://pypi.org/project/evidently/
- Evidently API `Report` и `DataDriftPreset`: https://evidentlyai.github.io/evidently/api-reference/v0.7.19/evidently.html
- Evidently presets API: https://evidentlyai.github.io/evidently/api-reference/main/evidently/presets.html
- scikit-learn classification metrics: https://scikit-learn.org/stable/api/sklearn.metrics.html